

UFUG 2602

栈与队列

Tengfei Chang, Jia Li, Xihu Zheng, Zijun Gong

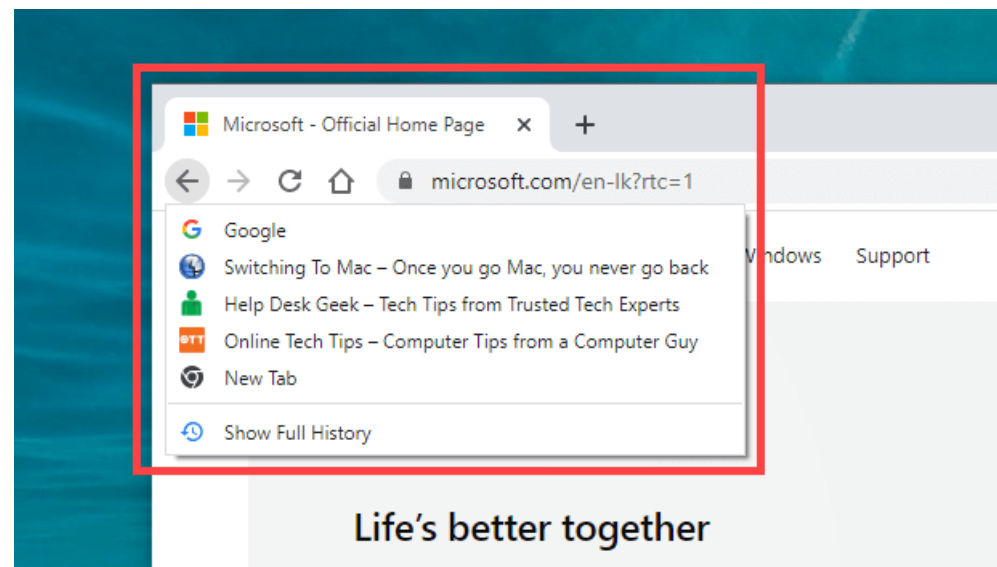
IoT, DSA, INTR Thrusts
香港科技大学 (广州)

大纲

- 栈
 - **栈的定义**
 - 栈的实现（数组 / 链表）
 - 括号匹配
- 队列
 - 队列的定义
 - 队列的实现

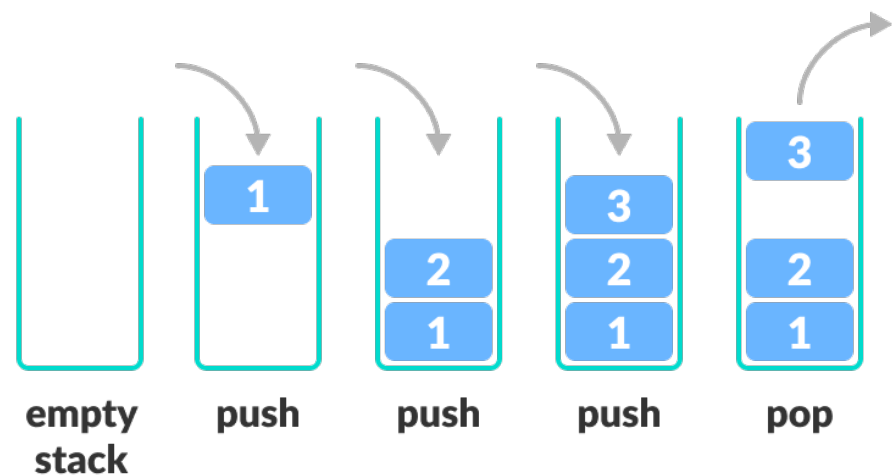
栈 - 示例

- 想象你正在浏览网页并点击不同页面。若要回到之前访问过的页面，可以：
 - 在地址栏输入上一页的 URL。
 - 点击“后退”按钮，沿已访问页面返回。
- “后退”功能帮助你按相同顺序回溯你访问页面的顺序。
- 这就是“后进先出”（LIFO）原则
它定义了栈这种数据结构！



栈

- 栈管理一个元素集合 S ，并支持两种操作：
 - $\text{push}(e)$: 将新元素 e 插入 S
 - pop : 从 S 中移除最近插入的元素并返回它（位于所谓的栈“顶”）。
- 元素的插入与删除遵循后进先出（LIFO）原则。



栈抽象数据类型

- 栈是支持两种核心操作的集合：
 - S.push(e): 将元素 e 压入栈顶。
 - S.pop(): 移除并返回栈顶元素；若栈为空则抛出错误。
- 便捷方法：
 - S.top(): 返回栈顶元素但不移除。
 - S.is_empty(): 若栈为空则返回 True。
 - len(S): 返回栈中元素个数。
- 关键性质：
 - 栈初始为空。
 - 容量不固定。
 - 元素可为任意类型。

Operation	Return Value	Stack Contents
S.push(5)	—	[5]
S.push(3)	—	[5, 3]
len(S)	2	[5, 3]
S.pop()	3	[5]
S.is_empty()	False	[5]
S.pop()	5	[]
S.is_empty()	True	[]
S.pop()	“error”	[]
S.push(7)	—	[7]
S.push(9)	—	[7, 9]
S.top()	9	[7, 9]
S.push(4)	—	[7, 9, 4]
len(S)	3	[7, 9, 4]
S.pop()	4	[7, 9]
S.push(6)	—	[7, 9, 6]
S.push(8)	—	[7, 9, 6, 8]
S.pop()	8	[7, 9, 6]

A series of stack operations and their effect on an initially empty stack S.

练习 1 — 非法操作

- 从空栈开始。执行：

```
push(5)
push(3)
pop()
push(2)
pop()
pop()
pop()
```

问题

1. 给出每次操作后的栈。
2. 最后一次 pop() 会发生什么？为什么？

练习 2 — 追踪 push/pop 序列

- 从空栈开始:

```
push (A)
push (B)
push (C)
pop ()
push (D)
pop ()
push (E)
```

问题

- 1.最终栈是什么（底 → 顶）？
- 2.pop() 返回了哪些元素？

大纲

- 栈

- 栈的定义
- **栈的实现（数组 / 链表）**
- 括号匹配

- 队列

- 队列的定义
- 队列的实现

基于数组的栈实现

- 实现栈 ADT 的一种简单方法是使用数组
- 我们从左到右添加元素
- 变量 t 记录栈顶元素的下标

算法 `size()`

return $t + 1$

算法 `pop()`

if *isEmpty()* then

throw *EmptyStackException*

else

$e \leftarrow S[t]$

$t \leftarrow t - 1$

return e

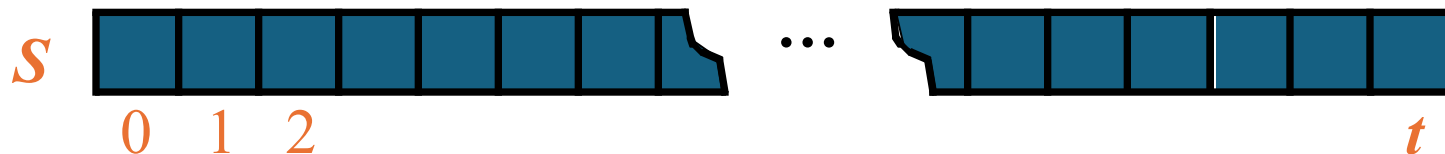


基于数组的栈实现

- 存储栈元素的数组可能已满
- 此时 `push` 操作将抛出 `FullStackException`
 - 基于数组实现的局限
 - 并非栈 ADT 本身固有
 - 通过扩容数组解决此问题

算法 `push(o)`

```
if  $t = S.length - 1$  then  
    throw FullStackException  
else  
     $t \leftarrow t + 1$   
     $S[t] \leftarrow o$ 
```



基于数组的栈实现

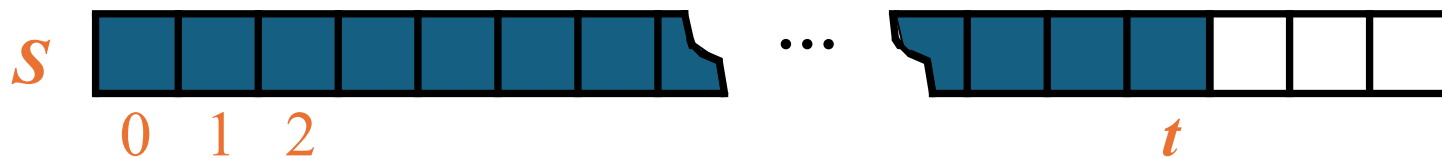
- 栈可能为空
- `top` 返回栈顶元素但不移除。若栈为空，应返回 `none`。

算法 `isEmpty()`

```
if  $t < 0$  then return true  
else return false
```

算法 `top()`

```
if isEmpty() then  
  throw EmptyStackException  
return  $S[t]$ 
```



基于数组的栈实现

- **时间复杂度:**

- **top()、is_empty()、len():** 最坏情况为常数时间 $O(1)$ 。
- **push() 与 pop():** 均摊 $O(1)$, 但因内部数组扩容, 最坏情况可达 $O(n)$ 。

- **空间复杂度:**

- 栈的空间占用为 $O(n)$ 。

- **效率提示:**

- 若已知最大规模, 预先将列表初始化为长度 n 比从空列表开始逐个追加更高效。
- 两种方法空间均为 $O(n)$, 但预分配在实践中更快。

Operation	Running Time
<code>S.push(e)</code>	$O(1)^*$
<code>S.pop()</code>	$O(1)^*$
<code>S.top()</code>	$O(1)$
<code>S.is_empty()</code>	$O(1)$
<code>len(S)</code>	$O(1)$

*amortized

Performance of our array-based stack implementation.

练习 3 — 实现栈 ADT (答案)

- 一种正确的 Python 列表实现:

练习 4 — 数组栈的扩容

- 假设栈初始容量 $\text{capacity} = 1$ 。
满时 $\rightarrow$ 容量加倍。

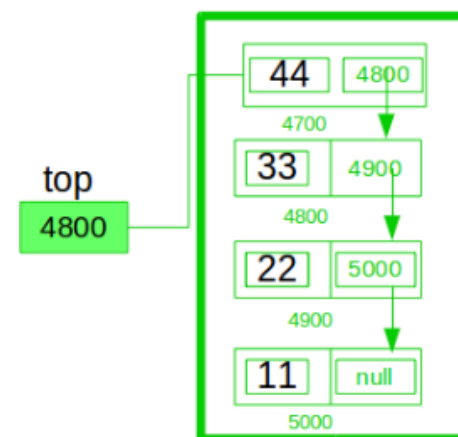
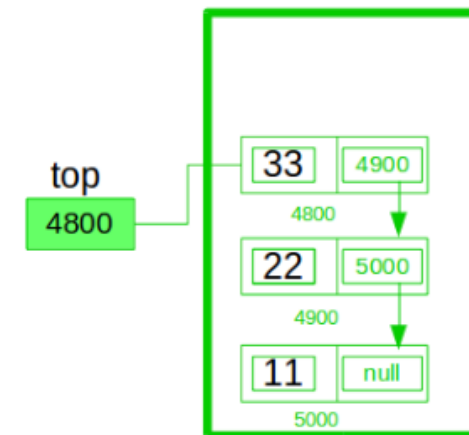
- 执行: `push 1...8`

问题

1. 每次扩容后的容量分别是多少?
2. 共发生几次扩容?

基于链表的实现

- 使用单向链表实现栈：
 - 将 S 的元素存放在单向链表 L 中。
 - 始终用 top 指针指向第一个结点
 - 入栈和出栈都发生在链表头部。
- 为什么用链表？
 - 它可以根据需要方便地扩大或缩小。



基于链表的栈实现

栈操作:

- push(): 向栈中插入一个新元素
 - 即只需在 L 的开头插入一个新元素。
- pop(): 返回栈顶元素
 - 即简单地返回并删除 L 的第一个元素。
- top(): 返回 L 的第一个元素。
- isEmpty(): 返回表示 L 是否为空的布尔值。

基于链表的栈实现

Push 操作:

- 初始化一个结点
- 将该结点的值设为 data, 即 `node.data = data`
- 将该结点插入链表头部
- 将 `top` 指针更新为当前结点

Pop 操作:

- 先检查链表中是否有结点, 若无则返回
- 否则令指针 `temp` 指向栈顶结点, 并将 `top` 前移一步
- 释放该 `temp` 结点

```
# Class representing a node in the linked list
class Node:
    def __init__(self, new_data):
        self.data = new_data
        self.next = None
```

```
# Function to push an element onto the stack
def push(self, new_data):

    # Create a new node with given data
    new_node = Node(new_data)

    # Check if memory allocation for the new node failed
    if not new_node:
        print("\nStack Overflow")
        return

    # Link the new node to the current top node
    new_node.next = self.head

    # Update the top to the new node
    self.head = new_node
```

```
# Function to remove the top element from the stack
def pop(self):

    # Check for stack underflow
    if self.is_empty():
        print("\nStack Underflow")
    else:

        # Assign the current top to a temporary variable
        temp = self.head

        # Update the top to the next node
        self.head = self.head.next

        # Deallocate the memory of the old top node
        del temp
```

基于链表的栈实现

Top (peek) 操作:

- 检查是否存在结点, 若无则返回
- 否则返回链表栈顶结点的值

isEmpty 操作:

- 检查头结点是否为 none, 若不是则返回 False
- 否则返回 True

```
# Class representing a node in the linked list
class Node:
    def __init__(self, new_data):
        self.data = new_data
        self.next = None
```

```
# Function to return the top element of the stack
def peek(self):

    # If stack is not empty, return the top element
    if not self.is_empty():
        return self.head.data
    else:
        print("\nStack is empty")
        return float('-inf')
```

```
# Function to check if the stack is empty
def is_empty(self):

    # If head is None, the stack is empty
    return self.head is None
```

基于链表的栈实现

- **时间复杂度:**
 - `push()`、`pop()`、`peek()` 均为常数时间 $O(1)$ ，因为不需遍历链表。
- **空间复杂度:**
 - 栈的空间占用为 $O(n)$ 。

特性	基于数组的栈	基于链表的栈
内存	连续存储，固定大小	动态分配，按需增长
扩容	代价高（需复制元素）	N/A
时间复杂度	$O(1)$	$O(1)$
空间复杂度	$O(n)$	$O(n)$
适用场景	最大规模已知时	栈规模不可预知时

用栈实现数据反转

- 由于 LIFO 协议，栈可用作反转数据序列的通用工具。
- 示例：
 - 按逆序打印文件中的各行。
 - 用栈反转列表中的元素。
 - 反转栈中元素的存储顺序。

```
1 # reversing data using a stack
2 def reverse_file(filename):
3     ''' Overwrite given file with its content line-by-line reversed'''
4
5     S = ArrayStack()
6     original = open(filename)
7     for line in original:
8         S.push(line.rstrip('\n')) # we will re-insert newlines when writing
9     original.close()
10
11     # Now we overwrite with contents in LIFO order
12     output = open(filename, 'w') # reopening file overwrites original
13     while not S.is_empty():
14         output.write(S.pop() + '\n') # re-insert newline characters
15     output.close()
16
17 #####
18 file = open("initial.txt", 'w')
19 file.write("I am going home.\n")
20 file.write("Today is a holiday.")
21 file.close()
22
23 !cat initial.txt
24 print('\n\n')
25 reverse_file("initial.txt")
26 !cat initial.txt
```

```
I am going home.
Today is a holiday.
```

```
Today is a holiday.
I am going home.
```

练习 5 — 合法出栈序列

- 栈初始为空。
按此顺序 push 数字 1, 2, 3, 4, 5。
- 下列哪些出栈序列是可能的?
 - A. 5 4 3 2 1
 - B. 4 5 3 2 1
 - C. 4 3 5 1 2
 - D. 3 5 4 2 1

练习 6 — 入栈与出栈各多少次？

- 栈初始为空。
- 共执行 20 次栈操作。
结束时栈中有 4 个元素。
- 发生了多少次 push 操作？

练习 6 — 入栈与出栈各多少次？

- 栈初始为空。
- 共执行 20 次栈操作。
结束时栈中有 4 个元素。
- 发生了多少次 push 操作？

练习 7 — 执行过程中的最大栈规模

- 计算执行过程中的最大栈规模：

```
push A  
push B  
pop  
push C  
push D  
pop  
push E  
push F  
pop  
pop
```

练习 8 — 函数调用栈推理

- 某程序执行如下函数调用：

```
main()  
  A()  
    B()  
      C()
```

写出各函数结束执行的顺序。

大纲

- 栈
 - 栈的定义
 - 栈的实现（数组 / 链表）
 - **括号匹配**
- 队列
 - 队列的定义
 - 队列的实现

括号匹配

检查表达式中括号是否正确嵌套。

- 一个表达式，例如 $[(2+3)*x+5]*2$ 。
- 每个开括号 "("、"{" 或 "[" 必须用匹配的闭括号 ")"、"}" 或 "]" 闭合。
- 打开括号后，必须按相反顺序闭合。
- 每个闭括号都有对应的同类型开括号。
 - 正确：
 1. `()( ){([ )]}`
 2. `(( ( ){([ ( ]))})`
 - 错误：
 1. `)( ( ){([ ( ])}(`
 2. `{[ ]}`
 3. `([ ])`
 4. `(`

括号匹配

- **算法步骤：**
 - 用栈 S 从左到右扫描表达式。
 - 读到开括号时，将其 push 入栈。
 - 读到闭括号时，pop 并检查栈顶弹出的元素是否与该闭括号构成有效配对。
 - 扫描结束时，若栈为空，则表达式括号完全匹配。
- **时间复杂度：**
 - 对长度为 n 的表达式，共有 n 次 push 和 n 次 pop。
 - 总时间复杂度为 $O(n)$ 。
- **要点：**
 - 符号（分隔符）集合是固定的（例如 $()$, $[]$, $\{\}$ ），因此匹配操作为常数时间。

括号匹配 – 示例 1

• 输入：() (() [()])

i	X[i]	操作	栈	输出
0	(	Push (	(	
1	)	Pop (检验 (与 X[i] 是否匹配? YES		
2	(	Push (	(	
3	(	Push (	((	
4	)	Pop (检验 (与 X[i] 是否匹配? YES	(	
5	[	Push [	([	

括号匹配 – 示例 1

• 输入：() (() [()]

i	X[i]	操作	栈	输出
6	(	Push (	([(	
7	)	Pop (检验 (与 X[i] 是否匹配? YES	([	
8	]	Pop [检验 [与 X[i] 是否匹配? YES	(	
9	)	Pop (检验 (与 X[i] 是否匹配? YES		
		检验栈是否为空? YES		TRUE

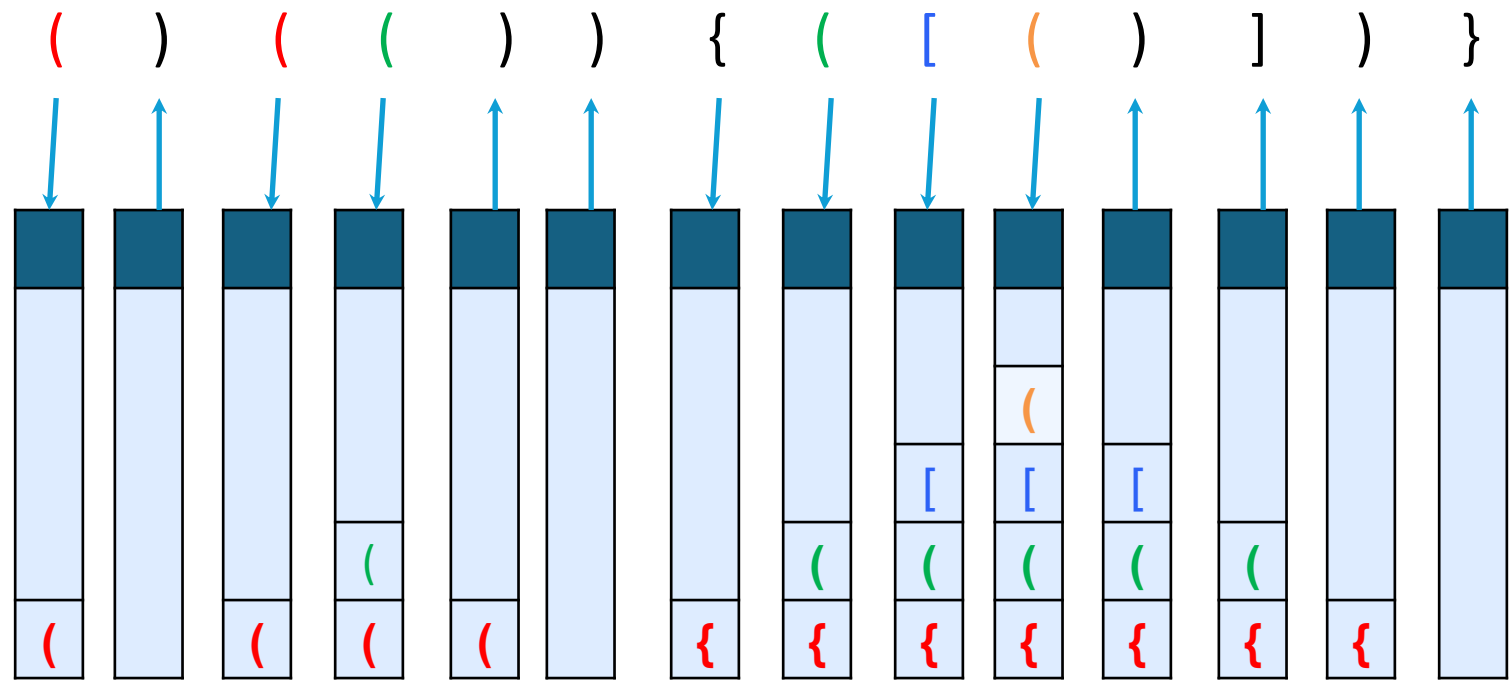
括号匹配 – 示例 2

• 输入：(([])

i	X[i]	操作	栈	输出
0	(	Push (	(	
1	(	Push (	((	
2	)	Pop (检验 (与 X[i] 是否匹配? YES	(	
3	[	Push [	([	
4	]	Pop [检验 [与 X[i] 是否匹配? YES	(	
5	]	Pop (检验 (与 X[i] 是否匹配? NO		FASLE

括号匹配

检查下列括号表达式：



括号匹配 – 算法

算法 ParenMatch(X,n):

输入: 含 n 个记号的数组 X , 每个记号为分组符号、
变量、算术运算符或数字

输出: 当且仅当 X 中所有分组符号均匹配时为 *true*

令 S 为空栈

for $i=0$ to $n-1$ **do**

if $X[i]$ 是开分组符号 **then**

$S.push(X[i])$

else if $X[i]$ 是闭分组符号 **then**

if $S.isEmpty()$ **then**

return false {无可匹配}

if $S.pop()$ 与 $X[i]$ 类型不匹配 **then**

return false {类型错误}

if $S.isEmpty()$ **then**

return true {全部符号已匹配}

else

return false {存在未匹配的符号}

练习 9 — 括号匹配 (答案)

- 判断下列各项是否合法:

({ [] })

([])

((()))

{ [() ()] }

(((

练习 10 — Min Stack (最小栈)

- 设计一个支持以下操作的栈：

```
push(x)  
pop()  
top()  
getMin() -> O(1)
```

- 示例：

```
push(3)  
push(5)  
push(2)  
getMin() -> 2  
pop()  
getMin() -> 3
```

提示：使用辅助栈。

大纲

- 栈

- 栈的定义
- 栈的实现（数组 / 链表）
- 括号匹配

- 队列

- **队列的定义**
- 队列的实现

队列 - 示例

假设你和朋友去买奶茶，店门口已经排了很长的队。你会怎么做？

1. 遵守规则，排到队尾，等前面的人都买完奶茶后才轮到你点单。
2. 插队直接到队首点单。

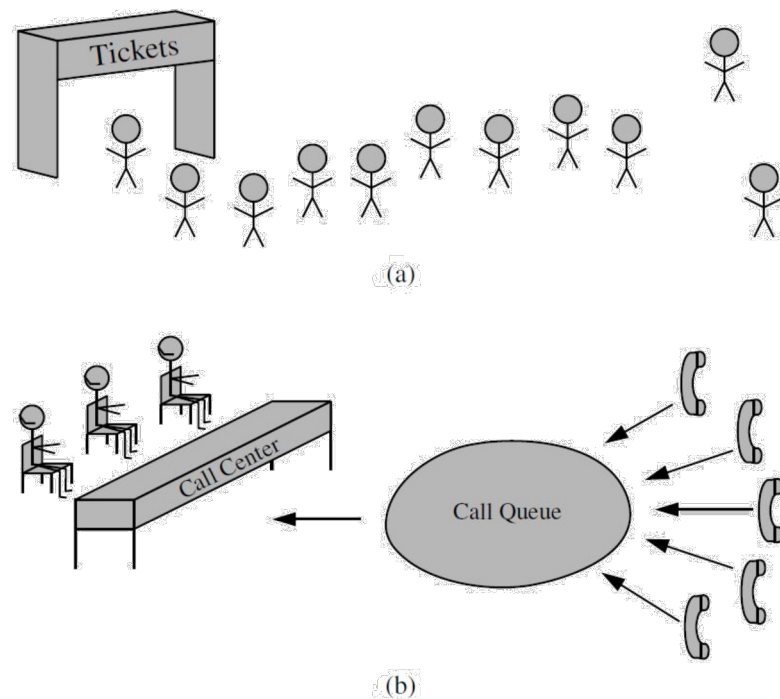
你多半会选第一种做法，因为排队遵循「先到先得」原则。先到的人先被服务，保证公平与秩序。

这种「先进先出」（FIFO）特性，正是队列（Queue）这一数据结构的本质！



队列

- 它是栈的近亲。
- 队列是一组对象的集合，按先进先出（FIFO）原则插入与删除。
- 元素可随时插入，但下一次只能删除在队列中停留最久的那个元素。
- 示例：
 - 排队等候游乐设施的游客。
 - 转接到客服中心的电话。



队列抽象数据类型

- 队列是遵循先进先出（FIFO）原则的抽象数据类型。它支持以下操作：
- 队列 ADT 对队列 Q 支持两个基本方法：
 - `enqueue(e)`：将元素 e 加入 Q 的队尾。
 - `dequeue()`：删除并返回 Q 中最早插入的（队首）元素。
- 附加方法：
 - `first()`：返回队列 Q 的队首元素，但不删除；
 - `is_empty()`：若队列 Q 不含任何元素则返回 `True`。
 - `len(Q)`：返回队列 Q 中的元素个数。

队列抽象数据类型

- 约定：新创建的队列为空。
- 队列容量事先没有上界。
- 加入队列的元素可以是任意类型。

Operation	Return Value	first $\leftarrow$ Q $\leftarrow$ last
Q.enqueue(5)	—	[5]
Q.enqueue(3)	—	[5, 3]
len(Q)	2	[5, 3]
Q.dequeue()	5	[3]
Q.is_empty()	False	[3]
Q.dequeue()	3	[]
Q.is_empty()	True	[]
Q.dequeue()	“error”	[]
Q.enqueue(7)	—	[7]
Q.enqueue(9)	—	[7, 9]
Q.first()	7	[7, 9]
Q.enqueue(4)	—	[7, 9, 4]
len(Q)	3	[7, 9, 4]
Q.dequeue()	7	[9, 4]

练习 1 — 基本 FIFO 跟踪

- 从空队列开始。执行：

```
enqueue (10)
enqueue (20)
dequeue ()
enqueue (30)
dequeue ()
enqueue (40)
enqueue (50)
dequeue ()
```

练习 2 — 操作计数

- 某队列共执行 25 次操作。
结束时含有 7 个元素。
- 发生了多少次 enqueue 和 dequeue?

练习 3 — 队列规模随时间变化

```
enqueue A  
enqueue B  
enqueue C  
dequeue  
dequeue  
enqueue D  
enqueue E  
enqueue F  
dequeue
```

- 最大规模?
- 最终队列?

大纲

- 栈
 - 栈的定义
 - 栈的实现（数组 / 链表）
 - 括号匹配
- 队列
 - 队列的定义
 - **队列的实现**

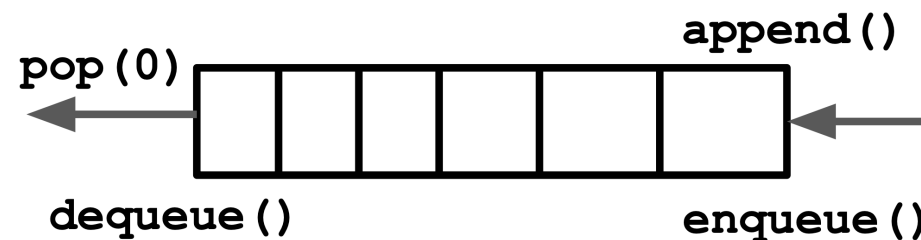
基于链表的实现

- 将 S 的元素存储在链表 L 中。
- En-queue(e): 将 e 插入到 L 的末尾。
- De-queue: 删除 L 的头结点并返回其中的元素。
- 始终跟踪头结点和尾结点的地址。
- Guarantees:
 - $O(n)$ 空间, where $n = |S|$ (假设 S 中每个元素占用 $O(1)$ 内存)。
 - 入队时间为 $O(1)$ 。
 - 出队时间为 $O(1)$ 。

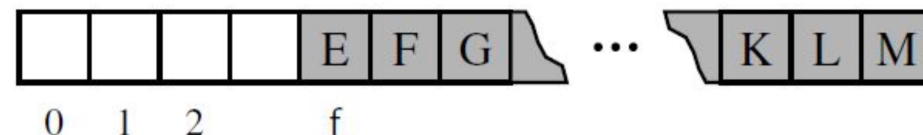
基于数组的队列实现 (1)

实现 #1

- 用 `append(e)` 将元素 `e` 加到数组末尾。
- 出队时用 `pop(0)` 删除列表的第一个元素。
 - 低效实现，时间为 $\Theta(n)$ 。



基于数组的队列实现 (2)

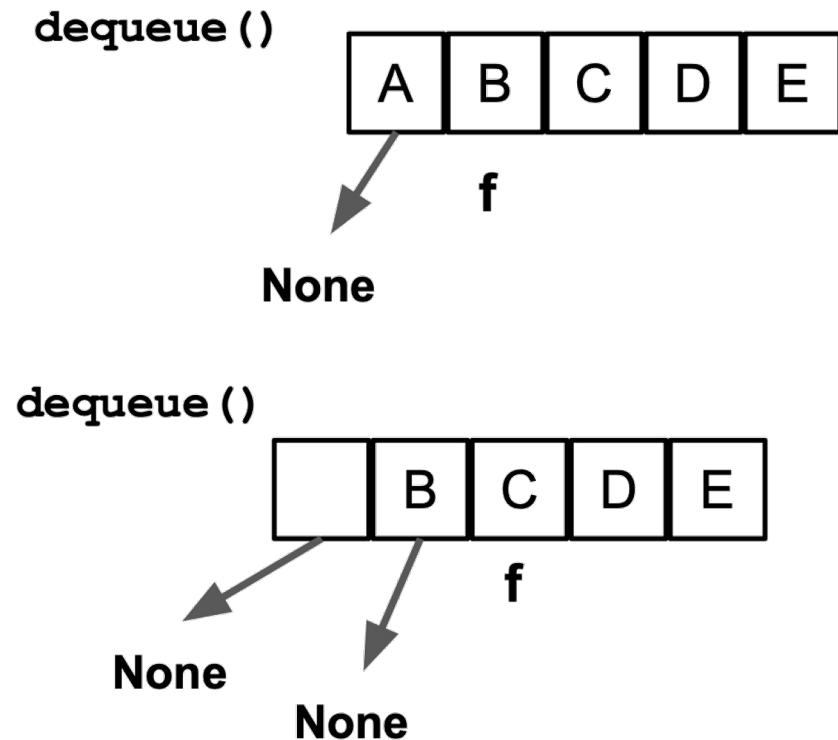


实现 #2

- 设 m 为 enqueue 操作的总次数。
- 分配大小为 m 的数组。
- 添加元素的方法相同。
 - 用显式变量 f 保存当前队首元素的下标。
 - 将数组中已出队位置替换为 `None` 引用。
 - **入队与出队时间: $O(1)$ 。**
 - **空间: $O(m)$, 即使实际元素个数 n 更小 ($m > n$)。队列可能占用比所需更多的空间。**

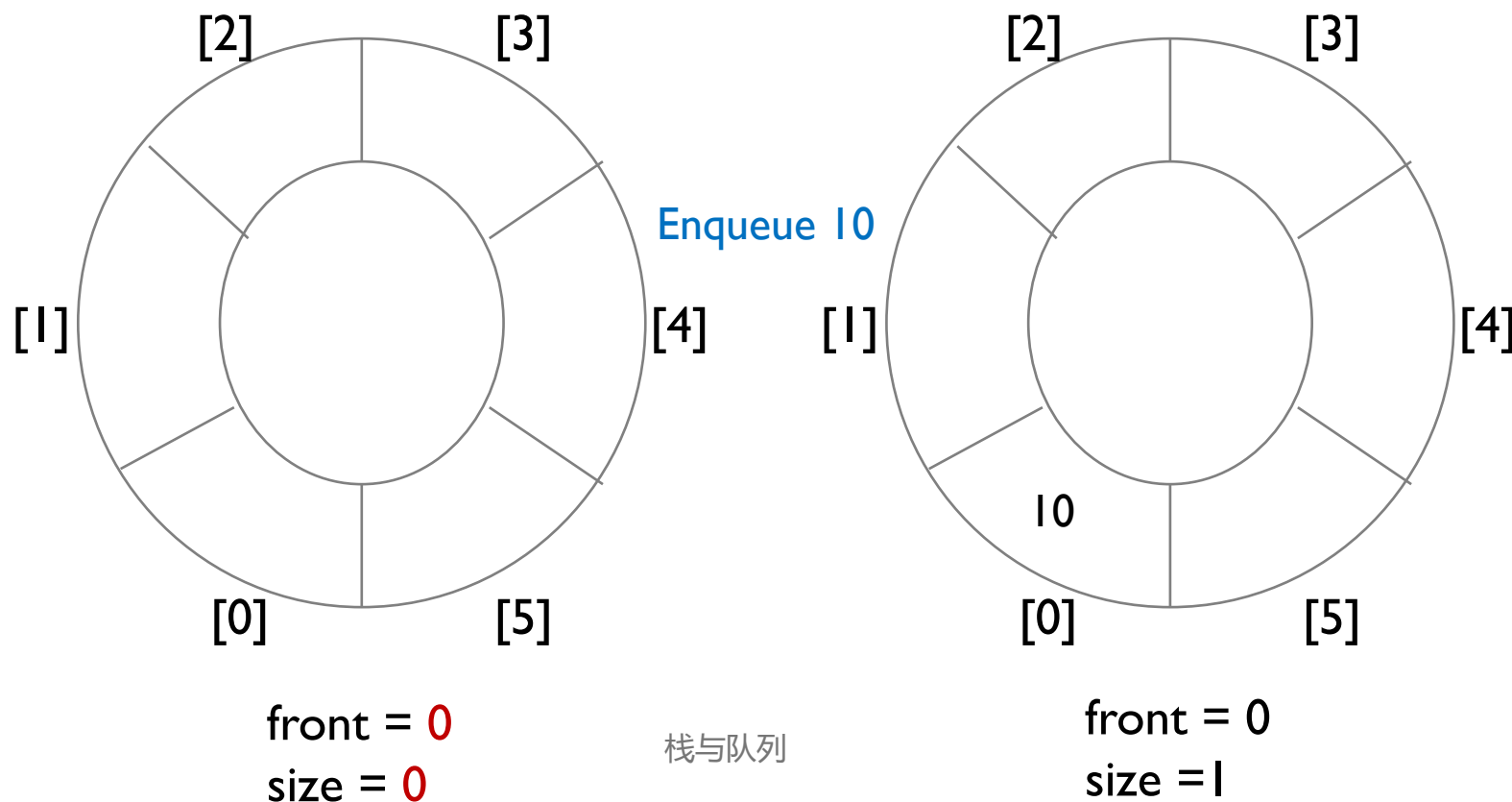
This design would have detrimental consequences in applications in which queues have relatively modest size, but which are used for long periods of time.

For example, the wait-list for a restaurant might never have more than 30 entries at one time, but over the course of a day (or a week), the overall number of entries would be significantly larger.

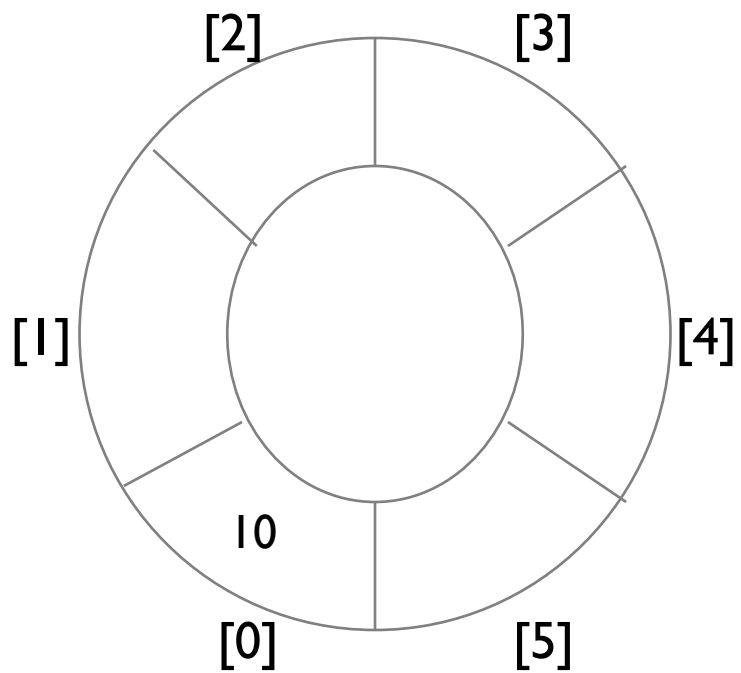


循环使用数组

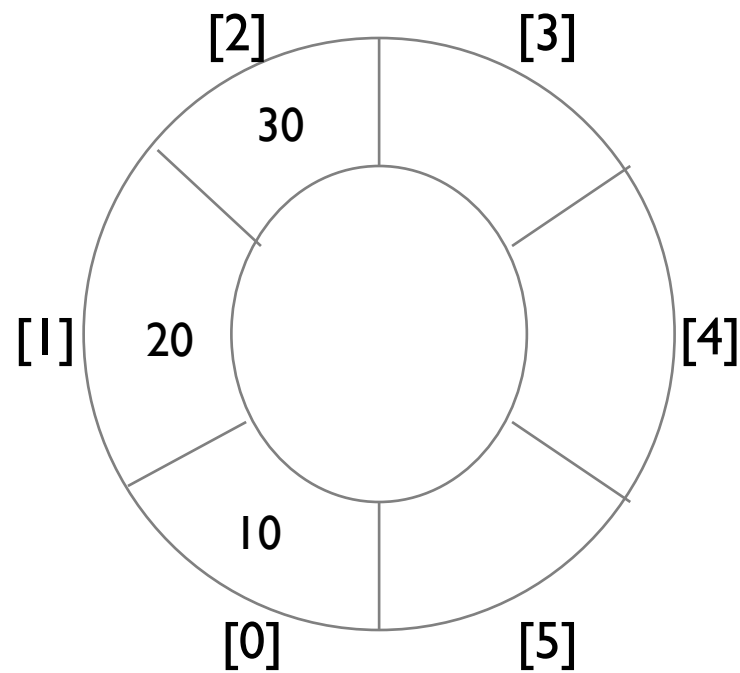
该实现使用长度为 N 的固定大小数组 (N 大于队列中的元素个数), 并允许队列在数组中环绕:



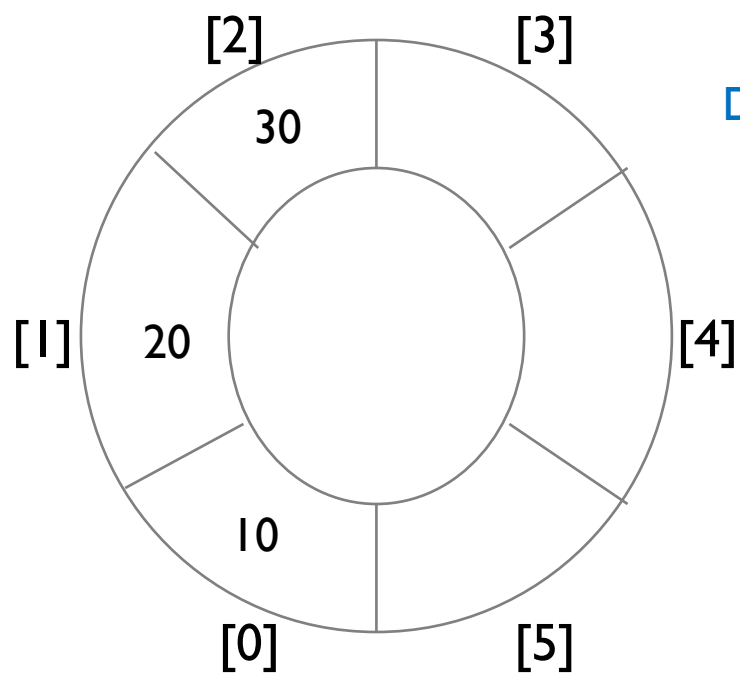
入队 20; 入队 30



front = 0
size = 1

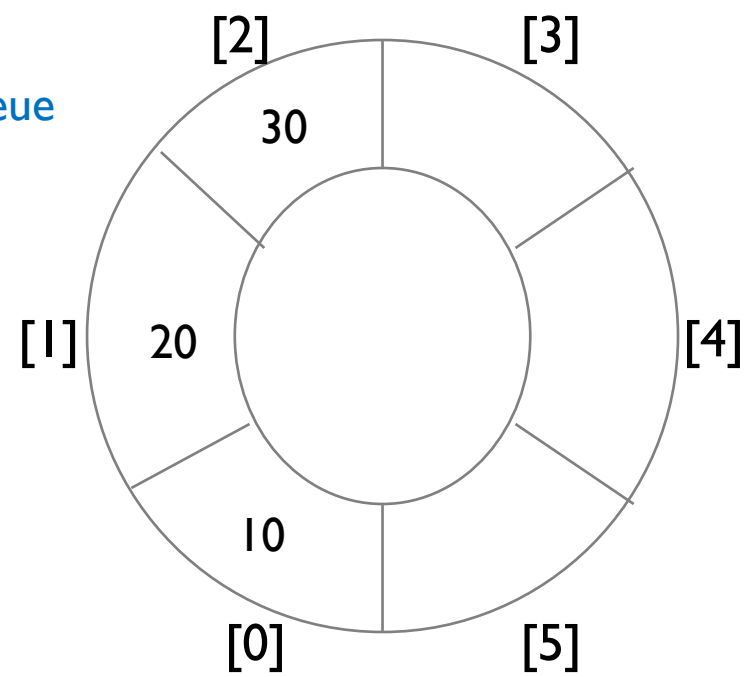


front = 0
size = 3



front = 0
size = 3

De-queue

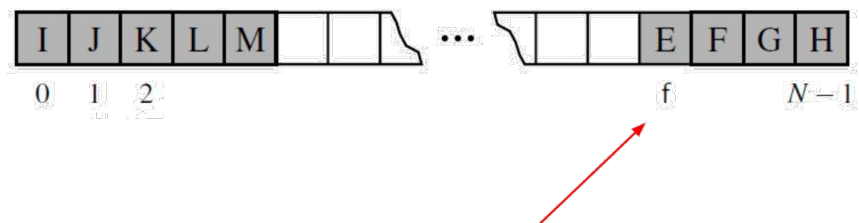


front = 1
size = 2

循环使用数组

该实现使用长度为 N 的固定大小数组（ N 大于队列中的元素个数），并允许队列在数组中回绕：

1. 队首 front 可以向右移动，元素可以从数组末尾回绕到开头。
2. 新元素加在队列的「尾」：从队首方向推进到下标 $N-1$ ，再从下标 0 继续。
3. 该做法通过减少搬移元素的开销，保证出队/入队时间为 $O(1)$ 。



Python 队列实现

- queue 类使用三个实例变量：
 - `_data`: 固定容量的列表, 存放队列元素。
 - `_size`: 队列当前元素个数。
 - `_front`: 队列首元素的下标。
- 访问方法:
 - `len()`: 返回当前队列长度。
 - `is_empty()`: 检查队列是否为空。
 - `first()`: 返回队列的队首元素。
- 修改方法:
 - `enqueue()`: 向队列添加一个元素。
 - `dequeue()`: 从队列删除一个元素。

```
1 class ArrayQueue:
2     '''
3     FIFO Queue implementation using a Python List as underlying storage
4     '''
5     DEFAULT_CAPACITY = 5      # moderate capacity for all new queues
6
7     def __init__(self):
8         ''' Create an empty queue '''
9         self._data = [None] * ArrayQueue.DEFAULT_CAPACITY
10        self._size = 0
11        self._front = 0
12
13
14    def __len__(self):
15        ''' return the number of elements in the queue'''
16        return self._size
17
18    def is_empty(self):
19        ''' Return True if the queue is empty'''
20        return self._size == 0
21
22    def first(self):
23        '''
24        Return (but do not remove) the element at the front of the queue
25        Raise Empty Exception if the queue is empty.
26        '''
27        if self.is_empty():
28            raise Empty('Queue is Empty')
29        return self._data[self._front]
```

```

32 def dequeue(self):
33     '''
34     remove and return the first element of the queue.
35     raise Empty exception if the queue is empty.
36     '''
37     if self.is_empty():
38         raise Empty('Queue is Empty')
39     answer = self._data[self._front]
40     self._data[self._front] = None # help garbage collection
41     self._front = (self._front + 1) % len(self._data) # circular indexing
42     self._size -= 1 # reduce the queue size
43     return answer
44
45 def enqueue(self, e):
46     '''Add an element to the back of queue'''
47     if self._size == len(self._data):
48         self._resize(2*len(self._data)) # double the array size
49     avail = (self._front + self._size) % len(self._data)
50     self._data[avail] = e
51     self._size += 1
52
53
54 def _resize(self, cap):
55     '''resize to a new list of capacity >= len(self)'''
56     old = self._data
57     self._data = [None] * cap
58     walk = self._front
59     for k in range(self._size): # only consider existing elements
60         self._data[k] = old[walk] # intentionally shift indices
61         walk = (1+walk) % len(old) # use old size as modulus
62     self._front = 0 # front has been realigned.
63
64 def __str__(self):
65     '''string representation of the queue'''
66     return '<'+''.join(str(self._data))+'>'

```

```

69 #####
70
71 Q = ArrayQueue()
72 Q.enqueue(5)
73 Q.enqueue(7)
74 Q.enqueue(9)
75 Q.enqueue(2)
76 Q.enqueue(6)
77 Q.enqueue(4)
78 Q.enqueue(1)
79 Q.enqueue(0)
80 |
81 print('Q: ', Q)
82 print('Queue Length:', len(Q))
83 print('Remove last item: ', Q.dequeue())
84 print('Remove last item: ', Q.dequeue())
85 print('Q: ', Q)
86 print('Queue Length:', len(Q))
87
88
89

```

Q: <[5, 7, 9, 2, 6, 4, 1, 0, None, None]>

Queue Length: 8

Remove last item: 5

Remove last item: 7

Q: <[None, None, 9, 2, 6, 4, 1, 0, None, None]>

Queue Length: 6

添加与删除元素

- 下一个元素加入队列的下标:

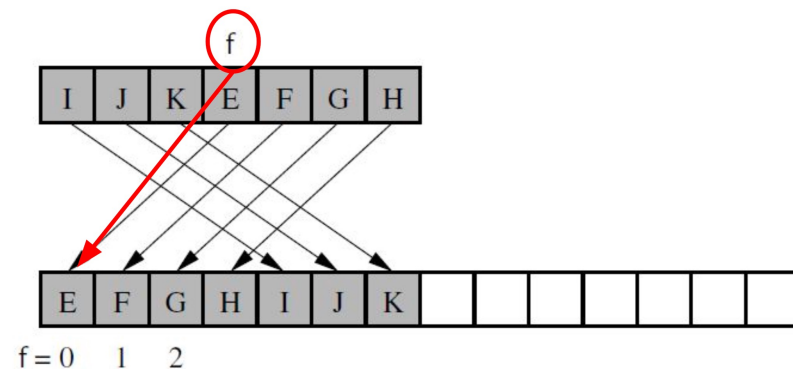
`avail = (self._front + self._size) % len(self._data)`

- 出队时, 返回并删除 `self._front` 处的值。然后将 `_front` 更新为循环队列中的下一位置:

`self._front = (self._front + 1) % len(self._data)`

调整队列容量

- 当队列规模等于底层数组大小时，将底层列表的存储容量加倍。
- 将旧列表中的引用复制到新列表。
- 转移内容时，有意将队首对齐到新数组的下标 0。



收缩底层数组

- 稳健做法：当元素个数降到容量的四分之一以下时，将数组缩小为当前大小的一半。
- 空间占用： $\sim\Theta(n)$

```
def dequeue(self):
    """
    remove and return the first element of the queue.
    raise Empty exception if the queue is empty.
    """
    if self.is_empty():
        raise Empty('Queue is Empty')
    answer = self._data[self._front]
    self._data[self._front] = None # help garbage collection
    self._front = (self._front + 1) % len(self._data) # circular indexing
    self._size -= 1 # reduce the queue size

    if 0 < self._size < len(self._data) // 4: # shrink the array size by half
        self._resize(len(self._data)//2) # when queue size 1/4 of the
    return answer # total array capacity
```

```
75 Q = ArrayQueue()
76 Q.enqueue(5)
77 Q.enqueue(7)
78 Q.enqueue(9)
79 Q.enqueue(2)
80 Q.enqueue(6)
81 Q.enqueue(4)
82 Q.enqueue(1)
83 Q.enqueue(0)
84
85 print('Q: ', Q)
86 print('Queue Length:', len(Q))
87 print('Remove last item: ', Q.dequeue())
88 print('Remove last item: ', Q.dequeue())
89 print('Q: ', Q)
90 print('Queue Length:', len(Q))
91 print('Remove last item: ', Q.dequeue())
92 print('Remove last item: ', Q.dequeue())
93 print('Remove last item: ', Q.dequeue())
94 print('Remove last item: ', Q.dequeue())
95 print('Q: ', Q)
96 print('Queue Length:', len(Q))
97 print('Remove last item: ', Q.dequeue())
98 print('Q: ', Q)
99 print('Queue Length:', len(Q))
100
101
102
```

```
Q:  <[5, 7, 9, 2, 6, 4, 1, 0, None, None]>
Queue Length: 8
Remove last item: 5
Remove last item: 7
Q:  <[None, None, 9, 2, 6, 4, 1, 0, None, None]>
Queue Length: 6
Remove last item: 9
Remove last item: 2
Remove last item: 6
Remove last item: 4
Q:  <[None, None, None, None, None, None, 1, 0, None, None]>
Queue Length: 2
Remove last item: 1
Q:  <[0, None, None, None, None]>
Queue Length: 1
```

分析基于数组的队列实现

Operation	Running Time
Q.enqueue(e)	$O(1)^*$
Q.dequeue()	$O(1)^*$
Q.first()	$O(1)$
Q.is_empty()	$O(1)$
len(Q)	$O(1)$

*amortized

练习 4 — 循环队列下标

- 容量 = 5
初始: `front = 0, size = 0`

- 操作:
enqueue 10
enqueue 20
enqueue 30
dequeue
dequeue
enqueue 40
enqueue 50
enqueue 60

60 插入在哪个下标?

练习 5 — 用两个栈实现队列

- 用两个栈设计队列。

练习 6 — 模拟混合操作

- 给定：栈 s ，队列 q
- 二者初始均为空。
- 执行操作：

```
push(S, 1)
push(S, 2)
enqueue(Q, 3)
enqueue(Q, 4)
pop(S) → enqueue(Q, ?)
dequeue(Q) → push(S, ?)
push(S, 5)
enqueue(Q, 6)
pop(S) → enqueue(Q, ?)
```

最终 s 和 q 是什么样？

练习 7 — 判断栈排列是否可行

- 输入队列：[1,2,3,4]
目标输出：[2,1,4,3]
- 能否用一个栈生成该序列？

练习 8 — 用两个队列实现栈

- 用两个队列设计栈操作。

Push(x):

Pop():

Top():

练习 9 — 交错队列的两半

- 队列: [1,2,3,4,5,6]
- 用栈 + 队列重排为 [1,4,2,5,3,6]。

练习 10 — 计算后缀表达式

- 用栈求值：
- $3\ 4\ +\ 2\ *\ 7\ /$

双端队列（拓展）

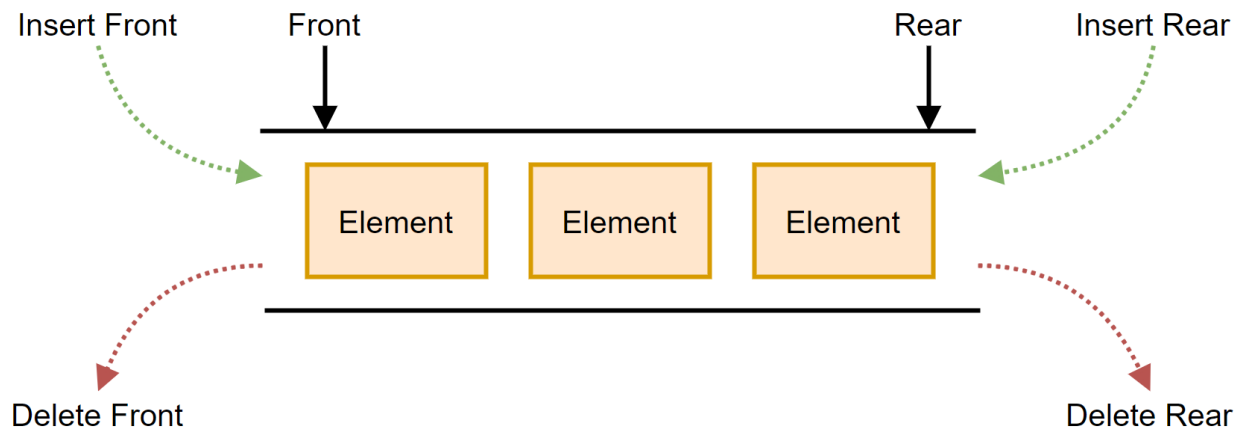
- Deque（双端队列）允许在两端进行插入和删除。
 - 兼具栈（LIFO）与队列（FIFO）的特性。

- **核心操作**

- `insertFront(x)` / `insertRear(x)`
- `deleteFront()` / `deleteRear()`
- `peekFront()` / `peekRear()`
- `isEmpty()`, `size()`

- **常见实现**

- 循环数组（环形缓冲区）
- 双向链表



滑动窗口最大值（拓展）

- 给定数组与窗口大小 k :
 - “窗口”从左 $\rightarrow$ 右滑动。
- 对每个位置，输出窗口内的最大值。

示例：

```
nums = [2,1,3,4,6,3,8,9,10,12,56]
```

```
k = 4
```

窗口与最大值：

```
[2 1 3 4] 6 3 8 9 10 12 56  $\rightarrow$  4
```

```
2 [1 3 4 6] 3 8 9 10 12 56  $\rightarrow$  6
```

```
2 1 [3 4 6 3] 8 9 10 12 56  $\rightarrow$  6
```

```
2 1 3 [4 6 3 8] 9 10 12 56  $\rightarrow$  8
```

```
2 1 3 4 [6 3 8 9] 10 12 56  $\rightarrow$  9
```

```
...
```

输出：

```
[4,6,6,8,9,10,12,56]
```

滑动窗口最大值（拓展）

- `nums = [1, 3, -1, -3, 5, 3, 6, 7]`
- `k = 3`
- 目标：对每个大小为 3 的窗口，输出最大值。

步骤 1 — 删除离开窗口的元素

步骤 2 — 从队尾删除更小的元素

步骤 3 — 将新元素加入队尾

步骤 4 — `Front` = 窗口最大值

逐步演示滑动窗口（拓展）

- **$i = 0$, $value = 1$:**

- 窗口尚未满。

`window = [1]`

`dq = [1]`

- **$i = 1$, $value = 3$:**

- 从队尾删除更小元素:
- $1 < 3 \rightarrow$ 删除 1

`window = [1, 3]`

`dq = [3]`

- **$i = 2$, $value = -1$:**

- 队尾无需删除
- $(3 > -1)$

`window = [1, 3, -1]`

`dq = [3, -1]`

`max = 3`

- **$i = 3$, $value = -3$**

- 离开窗口的元素 = 1;
- $1 \neq dq[0] \rightarrow$ 不删除;
- 队尾无需删除 $(-1 > -3)$

`window = [3, -1, -3]`

`dq = [3, -1, -3]`

`max = 3`

逐步演示滑动窗口（拓展）

- **i = 4, value = 5**
 - 离开的元素 = 3
3 == dq[0] → 删除队首
 - 接着从队尾删除更小元素：
 - -3 < 5 → 删除
 - -1 < 5 → 删除
- **i = 5, value = 3**
 - 离开的元素 = -1
 - -1 ≠ dq[0]
 - 从队尾删除更小元素：
 - 5 > 3 → 保留
- **i = 6, value = 6**
 - 离开的元素 = -3
 - -3 ≠ dq[0]
 - 从队尾删除更小元素：
 - 3 < 6 → 删除
 - 5 < 6 → 删除
- **i = 7, value = 7**
 - 离开的元素 = 5
 - 5 ≠ dq[0]
 - 从队尾删除更小元素：
 - 6 < 7 → 删除

```
window = [-1, -3, 5]
dq = [5]
max = 5
```

```
window = [-3, 5, 3]
dq = [5, 3]
max = 5
```

```
window = [5, 3, 6]
dq = [6]
max = 6
```

```
window = [3, 6, 7]
dq = [7]
max = 7
```

练习 11 — 滑动窗口最大值

```
nums = [4, 2, 2, 6, 1, 6, 3, 5]  
k = 3  
dq = []
```

对每个位置，输出窗口内的最大值。
并给出 dq 的内容